SDP 2025

User Design Document

Project Name	ShadowTrace
Faculty Supervisor Name	Dr. John Burns
Team Lead Name & ID	Said Mehraliyev 000014989
Team member #1 Name & ID • What sections did this person contribute to?	Said Mehraliyev 000014989
Team member #2 Name & ID What sections did this person contribute to?	Jamil Akbarov 000015227
Team member #3 Name & ID • What sections did this person contribute to?	Ibrahim Najafguliyev 000019888

Guidelines:

1. In Section 1 you should create a journey map in Figma, for **each of** the **four** most important use-cases from the research document. Do not worry about trivial use cases like user sign-up and user password reset.
2. In Section 2 (Database Design), discuss the data model you propose to support your user journey maps. The database design may be relational (using ERDs) or document storage (using document collections)
3. Page Count: 15 Pages Maximum

Section 1 User Journey Maps

HERE IS AN EXAMPLE

Use-case 1: Book a flight on the app

- Persona: Alex, 35, frequent traveler, wants a quick booking process.
- Goal: Book a round-trip flight.

Figma Implementation:

1. Canvas Setup: Create a horizontal timeline in Figma or FigJam with stages (Search Flights, Select Flight, Enter Details, Confirm Booking).
2. Visual Elements: Use rectangles for stages, sticky notes for actions (e.g., “Enters destination”), and a line graph for emotions (e.g., frustrated at slow search).
3. Touchpoints: Add icons for app screens, email confirmations, or payment gateways.
4. Pain Points: Note issues like “Slow flight searches due to unoptimized database queries.”
5. Collaboration: Share the file with the team; developers annotate database needs (e.g., “Index flight data for faster queries”).
6. Prototype: Link stages to wireframes (e.g., click “Search Flights” to view a search results screen).

Section 2: Database Design

In this section there are two choices: either choose a relational database design or a non-relational database design. In the rationale section, explain your reasoning.

Rationale

Here you should explain why you selected a Relational or a Non-Relational Database.

Relational Database Design

1. **Schema Structure**
    a. Entity-Relationship Diagrams (ERDs) for Figma Use Cases
    b. Tables and Relationships (e.g., Users, Roles, Transactions)
    c. Normalization to Ensure Data Consistency
2. **Query Optimization**
    a. Indexing for Fast Retrieval in Figma-Defined Flows (e.g., Search Queries)
    b. Joins and Aggregations for Data Display in UI
3. **Use Case Alignment**
    a. Supporting Structured Data Needs (e.g., Authentication, Reporting)
    b. Examples from Figma Prototypes (e.g., SQL Queries for User Profiles)

Non-Relational Database Design

4. **Schema Structure**
    a. Flexible Data Models (e.g., JSON Documents, Key-Value Stores)
    b. Collections for Dynamic Figma Use Cases (e.g., Product Catalogs, User Preferences)
    c. Denormalization for Performance in Unstructured Data
5. **Query Optimization**
    a. Aggregation Pipelines for Complex Data Retrieval
    b. Caching for Real-Time Interactions in Figma Prototypes
6. **Use Case Alignment**
    a. Supporting Scalable, Flexible Data (e.g., Real-Time Notifications, Analytics)
    b. Examples from Figma Journey Maps (e.g., MongoDB for User Activity Logs)

This appendix covers two design artefacts. Section 1 presents user journey maps for the four principal use cases of the ShadowTrace system. Section 2 sets out the database design, explains why a non-relational store was selected, and gives the resulting schema.

The two sections are connected deliberately. Each journey map contains at least one pain point whose cause lies in the data layer, and each of those is answered by a specific indexing or modelling decision in Section 2.

1.1 Method

Four actors were identified, one per journey: a developer who configures redaction, a security administrator who triages alerts, the detection system itself, and a compliance auditor. Each owns exactly one journey and appears in at least one other, so no actor is unaccounted for and no journey lacks an owner.

All four maps use an identical seven-row structure — Goals, Actions, Thoughts, Pain Points, Emotions, Touchpoints, Opportunities — across five stages, so the maps can be read comparatively rather than in isolation. The third journey substitutes Confidence for Emotions, because its actor is software; an emotional lane there would be fictional, whereas confidence is the property that actually governs whether the system is permitted to act.

Data sensitivity. A three-level classification runs through the maps: structural and temporal data only (service names, edge identifiers, timestamps, counts, entropy values); identifiers and metadata (attribute keys, policy configuration, user identifiers); and payload content (request bodies, authentication headers, database queries, free-text log fields). Payload content appears in exactly one place across all four journeys — the developer preview in Section 1.2 — and nowhere thereafter. That absence is the design argument made visible.

Coverage. The matrix below confirms responsibilities are distributed without gaps or duplication.

	Redaction config	**Alerting**	**Enforcement**	**Audit**
Developer	Owns	—	—	Supplies evidence
Administrator	—	Owns	Approves / notified	—
System	Applies policy	Generates alert	Owns	Records evidence
Auditor	Reviews	—	—	Owns

1.2 Developer Configures PII Redaction

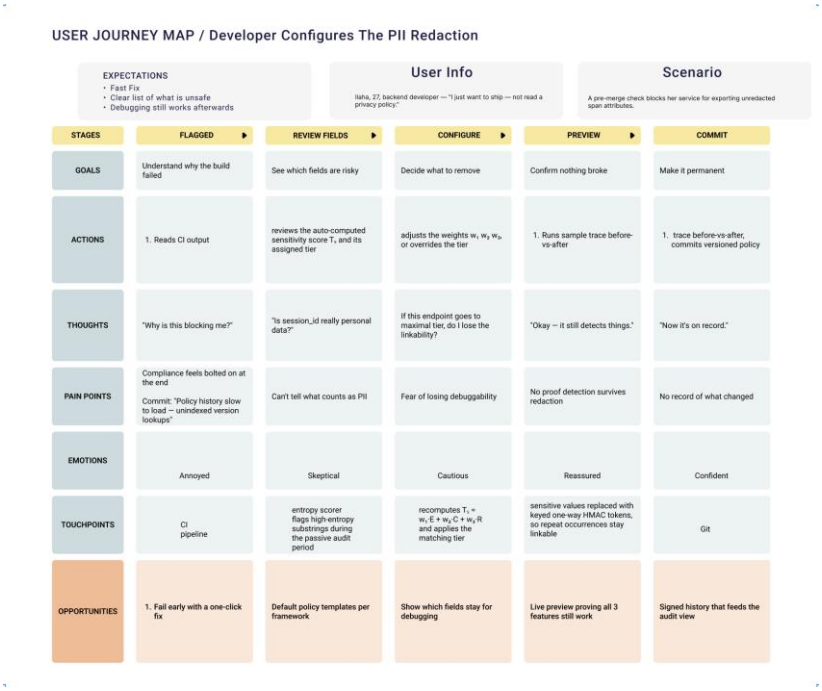


Figure B.1 — Developer configures PII redaction

This journey is where the project's central claim becomes a setting a person actually operates. Distributed tracing captures request bodies, headers and database queries by default, which is what turns a monitoring system into an uncontrolled copy of production data. GDPR Article 25 requires that minimization be architectural rather than a cleanup step, so the journey is framed as a pre-merge gate: a service that exports payload content cannot be shipped.

The developer does not label fields by hand. The system computes a per-endpoint sensitivity score from three signals — entropy-match rate, declared data classification, and blast radius — and assigns one of three redaction tiers automatically. The developer's task is to review that assessment, adjust the weighting, or override the tier. Sensitive values are then replaced with keyed one-way tokens rather than deleted, so repeated occurrences remain linkable within the call graph and the detector continues to function.

The Preview stage is the critical one. It is the only point in the entire system where payload content is visible, and only locally, so the developer can confirm with their own eyes that removing payload does not remove the detection features. Without that confirmation, developers assume redaction breaks their monitoring and disable it.

Data-layer pain point. At Commit, the signed policy history loads slowly because version lookups are unindexed. This is answered in Section 2.3 by a compound index on service name and version.

1.3 Administrator Views Lateral Movement Alert

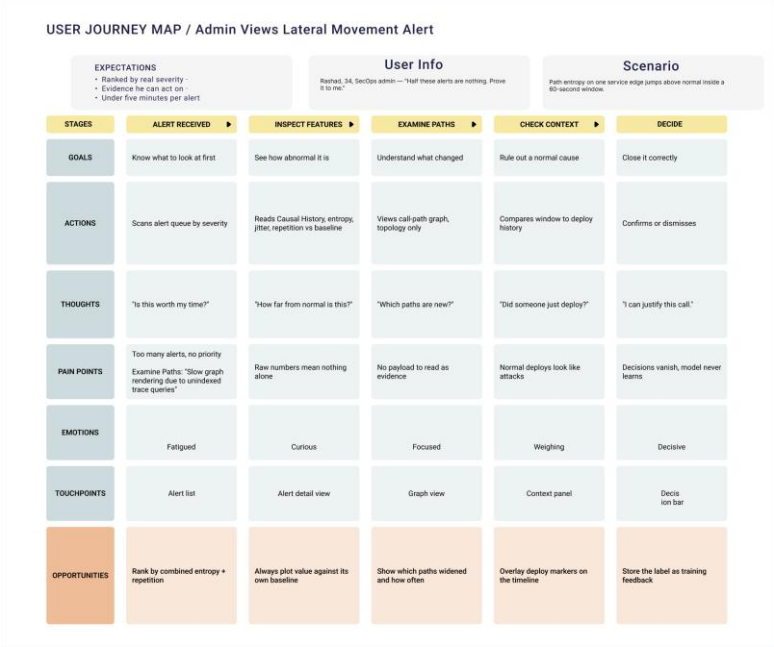


Figure B.2 — Administrator views lateral movement alert

This journey carries the research result into the product. Detection operates on four payload-free signals: path entropy, which measures how varied the routes through the system are; causal history, a graph-edit-distance comparison against a learned baseline that flags edges and nodes with no historical precedent; temporal jitter, which measures timing irregularity; and call repetition within a window.

The fourth signal exists because of a documented failure in the prior literature. A brute-force login attempt produces a request graph of two nodes and a single edge, structurally identical to a legitimate login, so any model scoring individual request graphs will classify it as normal. Repetition across a time window, rather than topology within one request, is what separates them. The alert view therefore presents all four signals together rather than collapsing them into a single score, because each answers a different failure mode.

The hardest design constraint appears at the Examine Paths stage. The administrator is accustomed to reading request payloads as evidence and can no longer do so, because that data was never stored. The interface compensates by making the system show its reasoning — which call paths appeared that normally do not, and how many times — so that structural evidence reads as evidence rather than as an unexplained number.

Data-layer pain point. At Examine Paths, graph rendering is slow because trace queries are unindexed. Section 2.3 answers this with an index on trace identifier.

1.4 System Auto-Enforces Istio Policy

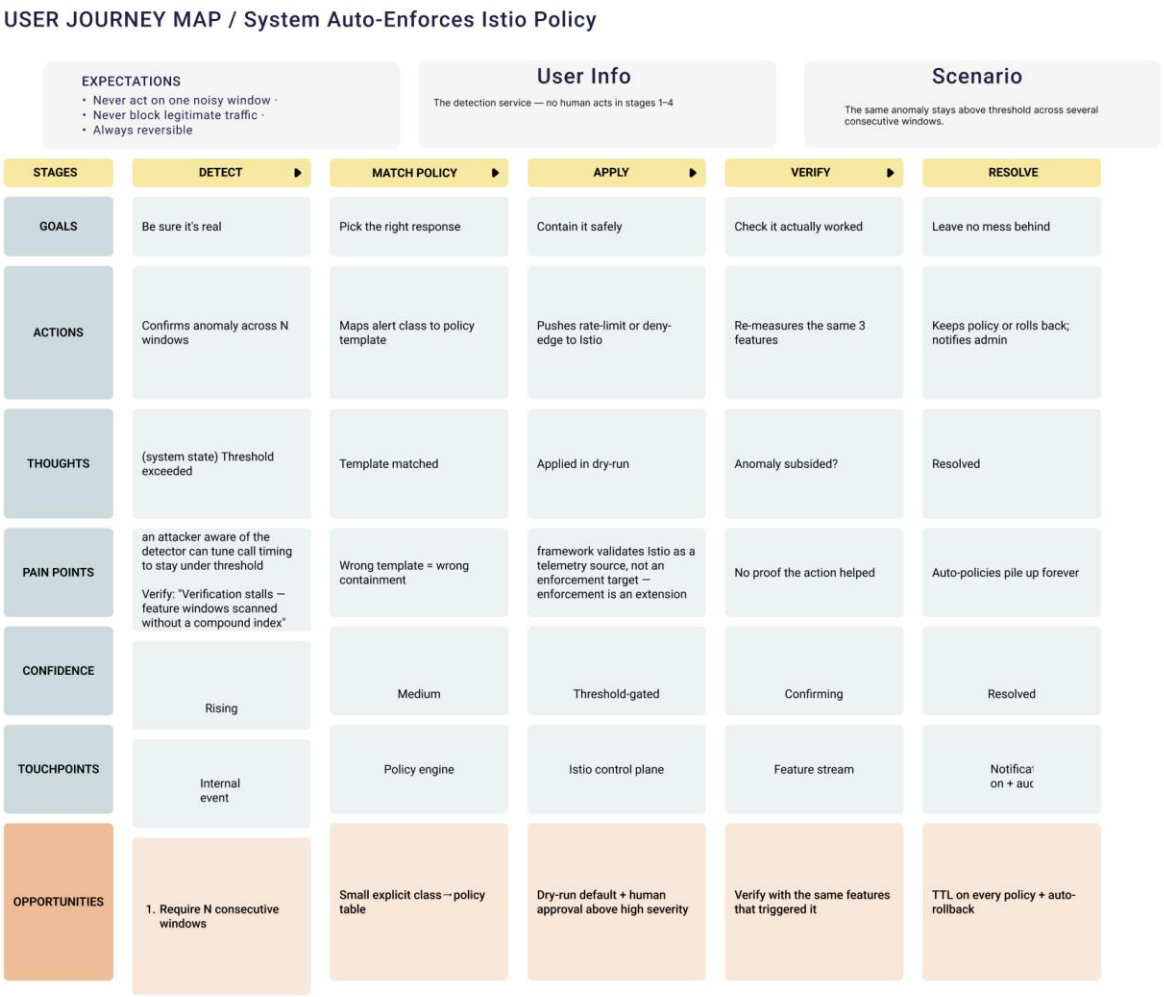


Figure B.3 — System auto-enforces Istio policy

The third journey has no human actor during its first four stages, which is why the emotional lane is replaced by confidence. The system detects an anomaly sustained across several consecutive windows, matches the alert class to a policy template, and pushes either a rate limit or an edge denial to the service mesh.

This is the most consequential journey in the design and is deliberately constrained. The research framework validates Istio as a telemetry source, not as an enforcement target, so automated blocking is an extension beyond what has been measured. Acting automatically on a detector whose accuracy remains an open question is how a security tool causes the outage it was built to prevent. Three brakes follow: dry-run mode is the default, human approval is required above a severity threshold, and every automatically applied policy carries an expiry.

The strongest element is the Verify stage. The system re-measures the same features that raised the alert and rolls the policy back if the anomaly has not subsided. It does not merely act; it establishes whether acting helped, which converts an assumption into a measurable claim. The Detect stage also records a known limitation: an adversary aware of the detection logic can pace calls to remain beneath threshold, so the results describe a non-adaptive attacker.

Data-layer pain point. At Verify, feature windows are scanned without a compound index, so verification stalls. Section 2.3 answers this with a compound index on trial identifier and window start.

1.5 Auditor Reviews Privacy Score

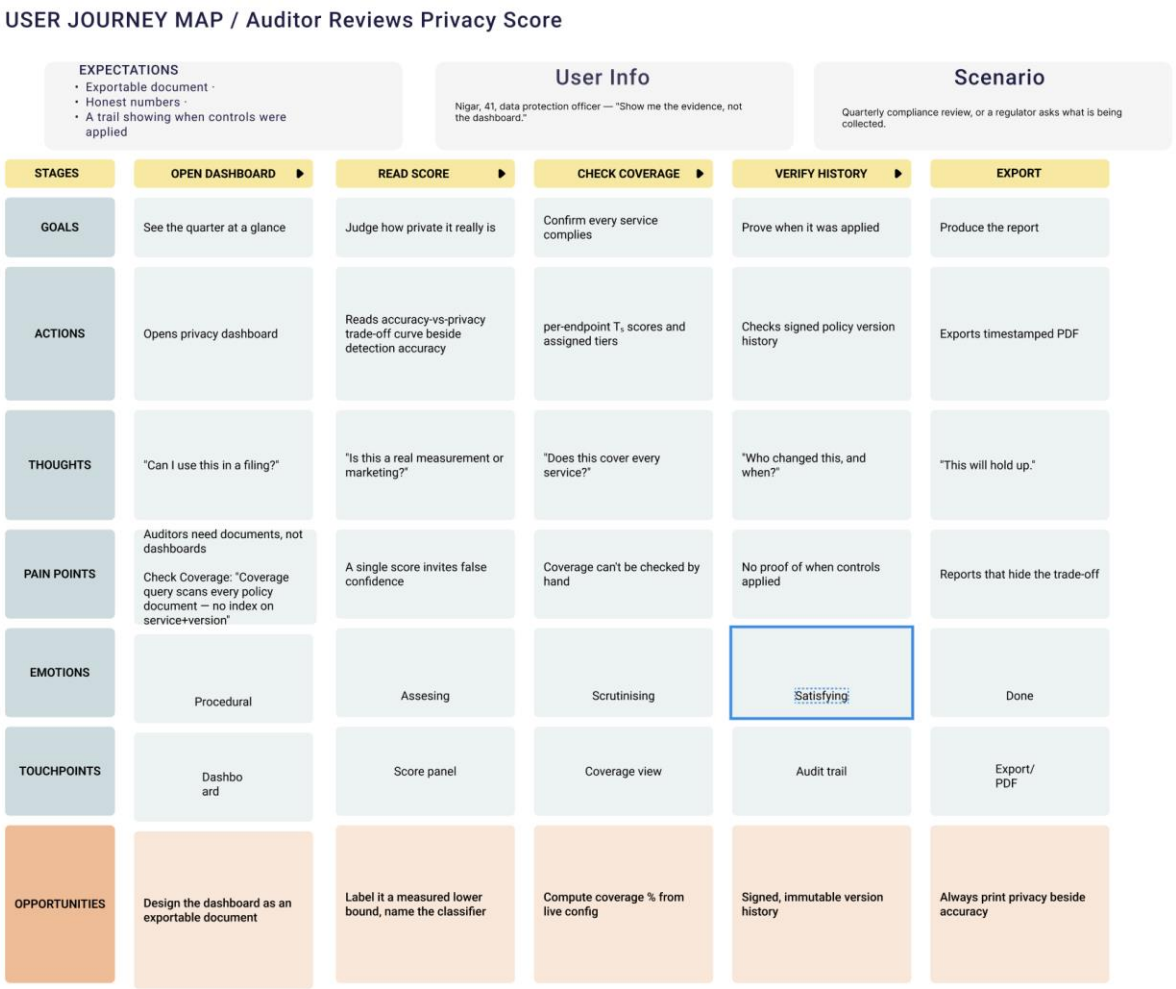


Figure B.4 — Auditor reviews privacy score

The fourth journey produces evidence rather than operational output. The auditor needs to demonstrate that minimization was designed in rather than retrofitted, which requires three things, all generated by earlier journeys: the signed policy history from Section 1.2 proves when controls were applied and by whom; the retention configuration proves that observation data does not persist beyond its purpose, satisfying Article 5(1)(e); and the privacy measurement provides a number.

That number is presented under two constraints. It is reported as a measured value with its method named, never as a guarantee, because a weaker measurement instrument would understate real exposure. And it is never displayed alone: the accuracy-versus-privacy trade-off is the project's central empirical contribution, so showing privacy without the detection accuracy it cost would conceal precisely the finding the work exists to produce.

The stage structure also reflects a practical observation about this actor. Auditors work from documents, not dashboards, so the interface is designed as an exportable artefact from the outset rather than as a screen with an export button added afterwards.

Data-layer pain point. At Check Coverage, the coverage query scans every policy document because no index exists on service and version. This is the same index that resolves the Section 1.2 pain point.

Section 2 — Database Design

2.1 Rationale

A non-relational document database, MongoDB, was selected over a relational design. Three arguments support this, given in order of weight.

The schema is the object of study. The feature vector is what the research tests. It moves from a twenty-five dimensional vector derived from logs and metrics to three privacy-safe features, with further variants across the planned ablations. Under a relational schema each variant is a migration with a fixed column set; under a document model it is a single field whose internal shape differs between records. Treating schema volatility as an experimental variable rather than a fixed contract is the decisive consideration.

The graphs are numerous and small, not few and large. The obvious challenge is why not a graph database such as Neo4j. Graph databases are optimised for deep traversal across one large connected graph. This project holds tens of thousands of separate request-level invocation graphs, some as small as two nodes and one edge, each read in its entirety. That is a document read rather than a traversal, and the shallow traversals that are required are served adequately by an aggregation stage.

The workload is write-heavy and time-bounded. Feature windows arrive continuously and are almost never updated. Time-series collections suit that shape, change streams push new alerts to the interface without polling, and expiry indexes provide automatic retention. The last of these is not merely housekeeping: it is the Article 5(1)(e) storage-limitation control enforced by the database itself rather than by a scheduled job.

Trade-offs accepted. A document store will not reject a malformed record unless validation rules are added explicitly; joins across collections are more awkward than in SQL; and multi-document transactions are available but are not the default. All three are acceptable here because the data is append-mostly and rarely joined across boundaries.

2.2 Schema Structure

Flexible data models

Records are stored as JSON-like documents, so fields may differ between records within the same collection. Two fields exploit this directly: the feature vector inside a graph document and the verification result inside a policy action are typed as free-form objects, and their internal shape changes with the experimental condition. Everything structural — identifiers, timestamps, edge keys, status values — remains fixed and indexed. A well-designed document schema is not schema-less; it pins down what is stable and permits flexibility only where the research requires it.

Collections for the journey map use cases

Eight collections are defined. Five map directly onto a journey: redaction policies to the developer journey, feature windows and alerts to the administrator journey, policy actions to the system journey, and privacy audits to the

auditor journey. Model versions records the retraining loop that the administrator's decisions feed. Trials and graphs hold the offline labelled data on which the detector is trained and benchmarked; no user interacts with them, and the live system operates without them.

Denormalization for performance

One rule governs the structure throughout. Children that are small and always read together with their parent are embedded within it, avoiding a join on the read path: the redaction rules inside a policy, and the nodes and edges inside a graph. Anything shared between parents, or expiring on a different schedule, is referenced instead. Alerts therefore hold window identifiers rather than copies of the windows, because a single window may trigger several alerts and because windows delete themselves on expiry while alerts must survive for audit.

Entity relationship diagram



Figure B.5 — Collection schema and relationships

Primary, foreign and unique key notation is shown for readability. MongoDB enforces only the document identifier; the remaining constraints are conventions maintained by the application. The trial identifier is marked unique because three collections reference it by its human-readable value rather than by its internal identifier, which keeps experimental records legible without a lookup.

Document structures

```
// redaction_policies
{
  "_id": "ObjectId",
  "service_name": "String",
  "version": "Number",
  "rules": [ { "attribute_key": "String", "action": "drop | hash | keep" } ],
  "features_enabled": ["String"],
  "sensitivity_score": "Number",
  "signals": {"E": "Number", "C": "Number", "R": "Number" },
  "tier": "maximal | coarse | minimal",
  "hmac_key_ref": "String",
  "author": "String",
  "committed_at": "ISODate",
  "is_active": "Boolean"
}
```

```
// feature_windows
{
  "_id": "ObjectId",
  "trial_id": "String",
  "edge_key": "String",
  "window_start": "ISODate",
  "window_end": "ISODate",
  "path_entropy": "Number",
  "entropy_baseline": "Number",
  "causal_history_distance": "Number",
  "temporal_jitter_z": "Number",
  "call_repetition": "Number",
  "repetition_baseline": "Number"
}

// alerts
{
  "_id": "ObjectId",
  "triggering_window_ids": ["ObjectId"],
  "alert_class": "lateral_movement | brute_force | probing",
  "severity": "low | medium | high",
  "edge_key": "String",
  "detected_at": "ISODate",
  "status": "open | confirmed | dismissed",
  "analyst_label": "String",
  "reviewed_by": "String",
  "reviewed_at": "ISODate"
}

// policy_actions
{
  "_id": "ObjectId",
  "alert_id": "ObjectId",
  "istio_policy_type": "rate_limit | deny_edge",
  "mode": "dry_run | awaiting_approval | enforced",
  "applied_at": "ISODate",
  "ttl_seconds": "Number",
  "verification_result": { "entropy_after": "Number",
                           "repetition_after": "Number",
                           "anomaly_subsided": "Boolean" },
  "rolled_back": "Boolean",
  "approved_by": "String"
}

// graphs
{
  "_id": "ObjectId",
  "trial_id": "String",
  "trace_id": "String",
  "timestamp": "ISODate",
  "nodes": [ { "node_id": "String", "service_name": "String" } ],
  "edges": [ { "from": "String", "to": "String",
               "latency_ms": "Number", "latency_zscore": "Number",
               "call_count": "Number" } ],
  "feature_vector": { "<flexible>": "Number" },
  "label": "String"
}

// trials
{
  "_id": "ObjectId",
  "trial_id": "String",
  "attack_type": "normal | http_flood | brute_force | sqli | ssrf",
  "platform": "String",
  "feature_condition": "String",
  "started_at": "ISODate",
  "ended_at": "ISODate",
  "split_assignment": "train | test"
}

// model_versions
{
  "_id": "ObjectId",
  "version": "Number",
  "trained_at": "ISODate",
  "positive_examples": "Number",
  "negative_examples": "Number",
  "macro_f1": "Number"
}

// privacy_audits
{
  "_id": "ObjectId",
  "feature_condition": "String",
  "run_at": "ISODate",
  "redaction_aggressiveness": "Number",
  "detection_macro_f1": "Number",
  "tier_coverage_pct": "Number",
  "redaction_coverage_pct": "Number"
}
```

2.3 Query Optimization

Indexing

Five indexes serve every screen described in Section 1. Each answers a pain point recorded on one of the journey maps.

```
db.feature_windows.createIndex({ trial_id: 1, window_start: 1 })
db.feature_windows.createIndex({ window_end: 1 }, { expireAfterSeconds: 604800 })
db.alerts.createIndex({ status: 1, severity: 1, detected_at: -1 })
db.graphs.createIndex({ trace_id: 1 })
db.redaction_policies.createIndex({ service_name: 1, version: -1 })
```

The compound index on trial and window start resolves the stalled verification in Section 1.4; the trace identifier index resolves the slow graph rendering in Section 1.3; the service and version index resolves both the slow policy history in Section 1.2 and the coverage scan in Section 1.5. The alert queue index supports sorting by severity and recency, which makes the five-minute triage target achievable. The expiry index is different in kind: it exists to

delete rather than to retrieve, and it is how the storage-limitation obligation is discharged automatically rather than by an operator remembering to run a cleanup task.

Aggregation pipelines

Two pipelines carry the analytical work. The baseline comparison behind the alert view matches an edge and time range, groups the matched windows to compute a rolling mean and standard deviation, and projects the deviation from that baseline — which is what allows a value to be shown against what is normal for that specific edge rather than a global threshold, the pain point recorded in Section 1.3. The second pipeline supports the quarterly report: it groups active policies by service, computes the proportion of endpoints covered at each tier, and joins the result to the most recent privacy audit so coverage and accuracy cost appear together.

Caching and real-time interaction

Rolling baselines are cached rather than recomputed per alert, since the underlying windows change far more slowly than alerts are opened. Change streams push new alerts to the interface without polling, keeping the queue current without repeated queries against a growing collection.

2.4 Use Case Alignment

The collections were derived from the journey maps rather than designed independently and mapped afterwards. Every collection exists because a specific actor or process requires something stored.

Collection	Purpose	Journey
redaction_policies	Versioned redaction configuration and tier assignment	Written in 1.2, read in 1.5
feature_windows	Four detection features per edge per window; self-expiring	Section 1.3
alerts	Detected anomalies and the analyst decision	Section 1.3
policy_actions	Automated responses, mode, expiry, verification	Section 1.4
privacy_audits	Privacy measurement paired with detection accuracy	Section 1.5
model_versions	Retraining records fed by analyst labels	Follows 1.3
trials	Experimental run and its train/test assignment	Research only
graphs	Labelled request-level invocation graphs	Research only

Scalable and flexible data. Alerts, feature windows and audit records have different shapes, arrive continuously, and are written far more often than read — the workload a document store handles well, and the practical reason the non-relational choice holds beyond the schema-volatility argument.

Flow. A committed policy determines what is collected; feature windows record the measurements; a window departing from its baseline raises an alert; a sustained alert produces a policy action that verifies itself against the same features; analyst decisions become labels for the next model version; and privacy audits read across the set to produce quarterly evidence. Trials and graphs sit alongside as the offline data the detector was trained and benchmarked on.